

数字电路设计第 2 讲

计算机组成概述

详细学习笔记

中南大学计算机学院软件工程系

主讲教师：任胜兵

2026 年 6 月 15 日

课程定位：本讲为数字电路设计的计算机组成入门章节，涵盖从计算机发展史到现代体系结构（冯·诺依曼结构 & 哈佛结构）、ARM 体系结构演进、SoC 设计理念，以及 CPU 性能指标体系（CPI、MIPS、执行时间、Amdahl 定律等）的核心概念。是本课程后续章节（指令系统、流水线、存储层次）的基础。

计算（机）的发展历程

四个时代划分

[p. p.3]

计算工具的发展可分为四个时代：

- 手工时代**——石头、手指、绳子等计数工具；商周时代的算筹。
- 机械时代**——算盘（公元前五世纪，中国）；对数计算尺（公元 17 世纪，欧洲）。
- 电子时代**——**ENIAC** (Electronic Numerical Integrator and Computer)，1946 年 2 月 15 日诞生，重 30 多吨，每秒 5000 次加法运算，是世界上第一台电子数字计算机。 [p. p.3]
- 量子时代**——2023 年中国发布“九章三号”光量子计算机原型（潘建伟院士团队），可操纵 255 个光子，打破可控光子数世界纪录。 [p. p.3]

中国计算机发展四代

[p. p.4]

1. **第一代电子管(1958–1964)**: 1958 年 8 月 1 日我国第一台电子数字计算机 103 型 (DJS-1 型, 仿制苏联) 诞生; 1964 年自行设计的 119 机研制成功。
2. **第二代晶体管(1965–1972)**: 1964 年 11 月研制成功第一台晶体管通用电子计算机 441-B 机。
3. **第三代中小规模集成电路 (1973–80 年代初)**: 1973 年研制成功 100 万次/秒大型通用计算机。
4. **第四代超大规模集成电路**: 1983 年 DJS-0520 微机 (与 IBM PC 兼容); 同年国防科技大学研制成功银河-I 巨型机, 运算速度上亿次/秒。

存储程序概念与冯·诺依曼结构

存储程序概念 (Stored Program Concept)

[p. p.5]

冯·诺依曼 (John von Neumann) 等人在 1946 年 6 月提出存储程序概念, 核心要点:

存储程序三大要点

1. 计算机硬件由**运算器、控制器、存储器、输入设备、输出设备** 5 大基本部件组成。
2. 计算机内部采用**二进制**表示指令和数据。
3. 将程序和原始数据**事先存入存储器**中, 再启动计算机工作。

这就是”存储程序”的基本含义——程序与数据统一存放在同一存储器中。

冯·诺依曼结构 (Von Neumann Architecture)

[p. p.9–10]

- 逻辑代码段和变量统一存储在**同一个内存**中, 按照代码执行顺序依次存储。
- 典型代表: ARM7 (ARM7TDMI 处理器)。
- 重要事实: 第一台计算机 ENIAC **不是**冯·诺依曼结构的计算机——因为其内存很小。

冯·诺依曼结构的瓶颈

由于指令和数据共享同一存储器和同一总线，CPU 不能同时取指令和取数据，形成了著名的“冯·诺依曼瓶颈” (Von Neumann Bottleneck)。这也是哈佛结构出现的动因。

计算机系统组成

计算机系统层次结构

[p. p.6]

计算机系统

- 硬件

- 主机

- * 中央处理器 (CPU) — 运算器 + 控制器

- * 内存储器 (主存)

- 外部设备—外存储器、输入设备、输出设备

- 软件

- 系统软件—操作系统、语言处理程序、服务性程序

- 应用软件—通用软件、用户程序

硬件组成框图

[p. p.7]

计算机硬件以系统总线 (System Bus) 为核心互联：

- CPU（控制器 + 运算器）通过系统总线与存储器、I/O 设备通信。
- 外围设备通过适配器（接口）接入总线。
- 主机 = CPU + 存储器。

树莓派 (Raspberry Pi) 实例

[p. p.8]

树莓派由英国慈善组织“Raspberry Pi 基金会”开发（埃本·阿普顿为项目带头人），2012 年 3 月正式发售。外形仅信用卡大小，但具备电脑所有基本功能，又称卡片式电脑，是学习计算机组成与嵌入式开发的理想平台。

哈佛结构 (Harvard Architecture)

核心特征

[p. p.11–12]

哈佛结构与冯·诺依曼结构相比有两个显著特点：

哈佛结构两大特点

1. 独立的存储模块：使用两个独立的存储器模块，分别存储指令和数据，每个模块不允许指令和数据并存。

2. 独立的总线：两条独立的总线，分别作为 CPU 与每个存储器之间的专用通信路径，两条总线之间毫无关联。

- 典型代表：ARM9 (ARM968 处理器)。
- 优势：可以同时取指令和取数据，避免冯·诺依曼瓶颈，适合 DSP 和实时嵌入式系统。

冯·诺依曼 vs 哈佛结构对比

特性	冯·诺依曼结构	哈佛结构
存储器	指令和数据共享同一存储器	指令和数据各独立存储器
总线	单总线	双总线（指令总线 + 数据总线）
并行取指/取数	不能（瓶颈）	能（并行）
典型代表	ARM7 (ARM7TDMI)	ARM9 (ARM968)
复杂度	较简单	较复杂
应用	通用计算机	DSP、嵌入式、微控制器

ARM 体系结构 (ARM Architecture)

ARM V9 架构（2021 年发布）

[p. p.14]

2021 年 ARM 推出全新一代 ARM V9 架构，主要提升：

- 安全性 (Security)：机密计算架构 (Confidential Compute Architecture, CCA)。
- AI 性能：矩阵乘法扩展 (Matrix Multiply Extension)。

- **整体性能速度：**较 V8 整体性能提升 30% 以上。
- **处理器 IP 核：**Cortex X2（性能核）、Cortex A710（效能核）、Cortex A510（效率核）。
- **注意：**Cortex X2 和 A510 仅支持 AArch64（64 位），A710 兼容 32 位。

ARM V10 架构（2023 年发布，前瞻验证中）

[p. p.15]

- **SVE2 (Scalable Vector Extension 2)：**向量处理原生支持。
- **FP16 半精度浮点：**原生支持半精度浮点运算。
- **MTE2 (Memory Tagging Extension 2)：**增强型内存标记扩展，提升内存安全。
- **SVE2 Crypto：**可扩展向量加密指令。
- 截至 2025 年末，该架构仍处于前瞻和验证阶段，**尚无量产芯片明确标注“ARMv10”认证标识**。市面所谓的“v10 就绪”多指 IP 核（如 Cortex 系列）已通过架构兼容性测试，而非物理芯片已流片上市。

SOC 片上系统 (System on a Chip)

SOC 组成

[p. p.16]

片上系统 (SoC) 将多个功能模块集成在一个芯片上，可能包含：

SoC 典型组成

- **CPU** (Central Processing Unit) —中央处理器
- **FPGA** (Field-Programmable Gate Array) —可编程逻辑单元
- **DSP** (Digital Signal Processor) —数字信号处理器
- **On-Chip Peripherals** 一片上外设（如 GPIO、UART、SPI、I²C、ADC 等）

ARM 的商业模式

[p. p.17]

ARM 公司本身不制造芯片，而是将 IP（知识产权）授权给其他半导体公司（如高通、苹果、三星、华为海思等），由授权方进行芯片设计和生产。这是一种”无晶圆厂” (Fabless) + IP 授权的商业模式。

ARM 公司发展简史

创立与早期

[p. p.18–19]

- **1978 年 12 月 5 日：**Hermann Hauser 和 Chris Curry 在英国剑桥创办 CPU 公司 (Cambridge Processing Unit)。
- **1979 年：**改名 Acorn 计算机公司。
- **1985 年：**Roger Wilson 和 Steve Furber 设计了第一代 32 位、6MHz 的处理器——ARM (Acorn RISC Machine)。
- **1990 年 11 月 27 日：**Acorn 改组为 ARM 计算机公司。苹果出资 150 万英镑，VLSI 出资 25 万英镑，Acorn 以 150 万英镑知识产权和 12 名工程师入股。办公室是一个谷仓。

爆发增长

[p. p.20]

- 21 世纪手机行业快速发展，ARM 处理器占领全球手机市场。
- 2006 年：全球 ARM 芯片出货量 20 亿片。
- 2010 年：ARM 合作伙伴出货量达 60 亿片。
- **2016 年 7 月 18 日：**日本软银集团以溢价 43%、总额 310 亿美元收购 ARM。
- 三大业务增长市场：移动计算（份额 > 85%）、企业基础架构（15%）、嵌入智能（25%）。
- 当前发力领域：服务器、汽车、物联网 (IoT)。

计算机的工作过程

指令执行循环 (Instruction Cycle)

[p. p.21]

计算机的工作实质是周而复始地取出指令、解释指令和执行指令的过程。具体步骤：

指令执行 5 步骤

1. 取指 (Fetch)：把程序计数器 PC 中的指令地址送存储器地址寄存器 AR，按该地址取出指令送指令寄存器 IR。

2. 取操作数：根据 IR 中的地址码，由地址计算部件形成操作数地址送存储器，取出数据送到运算器中的寄存器。

3. 译码 (Decode)：将 IR 中的操作码 OP 送指令译码器进行译码。

4. 执行 (Execute)：计算机有关部件在控制器发出的操作控制信号控制下，执行操作码 OP 规定的操作。

5. PC 更新：程序计数器 PC 加 1，形成下一条指令地址。

Fetch → Decode → Execute → PC+1 → Repeat

性能及主要技术指标

六大性能指标概览

[p. p.22]

1. 字长 (Word Length) 和主频 (Clock Frequency)
2. 存储容量 (Storage Capacity)
3. 运算速度 (Computing Speed)
4. 可靠性、可用性和可维修性 (RAS: Reliability, Availability, Serviceability)
5. 兼容性 (Compatibility) 和性价比 (Cost Performance)
6. CPU 性能评价方法 (Benchmark Methods)

字长和主频

[p. p.23]

字长 (Word Length) 计算机每次运算所包含的二进制位数，字长为 8 的倍数。字长标志着机器表示数的精度——位数越多，精度越高。

主频 (Clock Frequency) CPU 主时钟不断产生固定频率的时钟脉冲，该频率即主频。CPU 工作节拍由主频控制——主频越高，CPU 工作节拍越快，是影响计算机运行速度的重要参数。

存储容量 (Storage Capacity)

[p. p.24]

- **主存储器 (Main Memory):** CPU 可直接访问；容量指主存存储单元总数。如 16 位地址码的计算机，主存容量为 $2^{16} = 64K$ 字节。主存容量往往较小。
- **外存储器 (External Storage):** 联机外存容量，存放暂不参与运行的程序和数据。外存容量往往较大。

运算速度 (Computing Speed)

[p. p.25-27]

核心概念

CPI (Cycles Per Instruction) 每条指令执行所需的 CPU 时钟周期数。

IPC (Instructions Per Cycle) CPI 的倒数，每个时钟周期内执行的指令数： $IPC = 1/CPI$ 。

Fz CPU 工作时主时钟的固定频率。

MIPS (Million Instructions Per Second) 每秒执行的百万条指令数。

MFLOPS (Million Floating-point Operations Per Second) 每秒百万个浮点操作。

MIPS 计算公式

[p. p.26-27]

MIPS 核心公式

$$MIPS = \frac{\text{指令条数 (百万为单位)}}{\text{执行时间 (s)} \times 10^6} = \frac{Fz \times IPC}{10^6} = \frac{Fz}{CPI \times 10^6}$$

程序执行时间 $T = \frac{\text{指令数} \times \text{CPI}}{\text{主频}}$

计算示例

[p. p.27]

已知某程序各类指令的 CPI 和使用频率：

指令类型	CPI	使用频率
类型 1	1	60%
类型 2	2	18%
类型 3	4	12%
类型 4	8	10%

平均 CPI = $1 \times 60\% + 2 \times 18\% + 4 \times 12\% + 8 \times 10\% = 2.24$

若主频为 1GHz (10⁹Hz)，则：

$$\text{MIPS} = \frac{10^9}{2.24 \times 10^6} \approx 446.4 \text{ 每秒百万条指令}$$

若程序有 2 × 10⁵ 条指令，则执行时间：

$$T = \frac{2 \times 10^5 \times 2.24}{10^9} = 0.448 \text{ ms}$$

可靠性、可用性和可维修性 (RAS)

[p. p.28]

可靠性 (Reliability) 用故障平均间隔时间 **MTBF** (Mean Time Between Failures) 衡量：

$$\text{MTBF} = \frac{\text{系统有效使用时间}}{\text{故障次数}}$$

可用性 (Availability)

$$\text{可用性} = \frac{\text{系统有效使用时间}}{\text{系统有效使用时间} + \text{故障修复时间}}$$

可维修性 (Serviceability) 用故障修复平均时间 (MTTR, Mean Time To Repair) 来衡量。

兼容性和性价比

[p. p.29]

向上兼容 / 向前兼容 (Forward Compatibility) 在较低档计算机上编写的程序可以在同一系列的较高档计算机上运行。例如：Intel 386 上的软件可运行在更高档次机型上。

向下兼容 / 向后兼容 (Backward Compatibility) 程序/库更新到较新版本后，用旧版创建的文档仍能被正常操作。例如：Word 2007 可打开 Word 2003 的文件。

性价比 (Cost Performance) 计算机性能与其价格之间的比例关系，主要受处理器、内存、外部设备等硬件组件的性能和价格影响。

CPU 性能评价方法

[p. p.30]

两种方法

1. **CPU 执行时间方法**：直接测量执行时间，但忽略了 I/O 结构、操作系统等对系统性能的影响。
2. **基准程序测试方法 (Benchmark)**：利用能反映计算机典型运行情况的一组标准化程序进行测试，结果用于评价不同计算机的相对性能。

常用基准程序

基准程序	测试内容
Dhrystone	整型运算性能（常用、核心通用基准）
Linpack	浮点运算性能（主要用于工作站评测）
Whetstone	浮点运算性能（主要用于微型计算机）
SPEC (SPEC-ML)	全面反映机器性能，AI 算力评测标准
TPC	事务处理、数据库处理、企业管理与决策支持等

常用性能测试工具

[p. p.31]

- **CPU-Z** (免费): 检测处理器和主板型号，提供详细硬件信息。<https://www.cpuid.com/softwares/cpu-z.html>
- **Prime95** (免费): 公认最残酷的烤机软件, 评估 CPU 稳定性和性能。<https://sourceforge.net/projects/prime95/>

- **PerfDog** (收费: 充值): 腾讯开发, 跨平台性能测试, 覆盖 CPU、内存、GPU、帧率、电量、网络等。 <https://perfdog.qq.com/>

Amdahl 定律 (Amdahl's Law)

定义

Amdahl 定律是计算机体系结构中评估**系统加速比**的经典定律, 由 Gene Amdahl 于 1967 年提出。该定律指出: 对系统某一部分进行性能改进, 整个系统性能的提升程度取决于该部分在整体中的占比。

Amdahl 定律加速比公式

设可改进部分在系统中的时间占比为 f ($0 \leq f \leq 1$), 该部分的加速比为 S_f ($S_f > 1$), 则整个系统的加速比 S 为:

$$S = \frac{1}{(1-f) + \frac{f}{S_f}}$$

当 $S_f \rightarrow \infty$ (即改进部分被无限加速), 加速比的极限为:

$$S_{\max} = \frac{1}{1-f}$$

核心推论

1. **边际收益递减**: 即使对系统某一部分无限加速, 整体加速比受限于未被改进部分的比例。
2. **常见优化优先**: 应优先优化占据运行时间最大的部分, 以获取最大加速比。
3. **量化设计原则**: 计算机体系结构著名设计原则”Make the common case fast” 即来源于 Amdahl 定律。

计算示例

假设某程序有 40% 的部分可通过并行化加速 ($f = 0.4$), 并行部分的加速比为 10 倍 ($S_f = 10$):

$$S = \frac{1}{(1-0.4) + \frac{0.4}{10}} = \frac{1}{0.6 + 0.04} = \frac{1}{0.64} \approx 1.56$$

即: 尽管并行部分获得了 10 倍加速, 整体程序只加速了约 1.56 倍。如果并行部分无限加速 ($S_f \rightarrow \infty$), 极限加速比仅为 $S_{\max} = 1/0.6 \approx 1.67$ 倍。

与 CPI/MIPS 的关系

Amdahl 定律可推广至 CPU 性能优化场景：当优化某类指令的 CPI 时，整体 CPI 的改进受限于该类指令在指令流中的占比（即 MIPS 计算示例中的使用频率权重），这与平均 CPI 的加权计算思想一致。

考点总结与速查

核心考点速查表	
1. 冯·诺依曼结构三大要点：五大部件、二进制、存储程序。	[p. p.5]
2. 哈佛结构两大特点：独立存储模块（指令/数据分离）、独立总线。	[p. p.11]
3. 冯·诺依曼 vs 哈佛：单总线/双总线、共享存储器/独立存储器、通用/DSP。	[p. p.9–12]
4. ARM V9 三大提升：安全性、AI 性能、整体速度。	[p. p.14]
5. ARM V10 关键特性：SVE2、FP16、MTE2、SVE2 Crypto（截至 2025 仍无量产芯片）。	[p. p.15]
6. SoC 组成：CPU + FPGA + DSP + On-Chip Peripherals。	[p. p.16]
7. 指令执行五步：取指 → 取操作数 → 译码 → 执行 → PC+1。	[p. p.21]
8. CPI 定义：Cycles Per Instruction，每条指令所需时钟周期数。	[p. p.26]
9. MIPS 公式： $MIPS = F_z / (CPI \times 10^6)$ 。	[p. p.26]
10. 执行时间公式： $T = \text{指令数} \times CPI / \text{主频}$ 。	[p. p.27]
11. Amdahl 定律： $S = 1 / [(1 - f) + f / S_f]$ ，极限加速比 $S_{\max} = 1 / (1 - f)$ 。	
12. MTBF：系统有效使用时间 / 故障次数。	[p. p.28]
13. 向上兼容：低档程序可在高档运行；向下兼容：新版可打开旧版文档。	[p. p.29]

课后自测题

- 1. 简述冯·诺依曼结构存储程序概念的三大要点。
- 2. 对比冯·诺依曼结构和哈佛结构的主要区别（至少三点）。

3. 什么是 SoC? 典型 SoC 包含哪些模块?
4. 写出 MIPS 的计算公式, 并说明 CPI、IPC、Fz 的含义。
5. 某程序平均 $CPI=2.24$, 主频 $1GHz$, 含 2×10^5 条指令, 求 MIPS 和执行时间。
6. 写出 Amdahl 定律的加速比公式。若程序 30% 可并行化且该部分加速 8 倍, 求整体加速比。
7. 解释 MTBF 的定义和计算公式。
8. 向上兼容和向下兼容各举一个例子。

中南大学计算机学院 • 数字电路设计课程 • 复习资料

制作日期: 2026 年 6 月 15 日 来源: 任胜兵老师课件《计算机组成概述》